



使用NIO改造网络聊天室

北京理工大学计算机学院
金旭亮

原有聊天室存在的性能问题

BIO版的聊天室中，以下几个地方严重地影响Server性能：



`ServerSocket.accept()`：阻塞式等待客户端连接



`InputStream.read()`、`OutputStream.write()`：
阻塞式流操作



无法在同一个线程里处理多个Stream I/O

非阻塞式的NIO



用Channel代替Stream

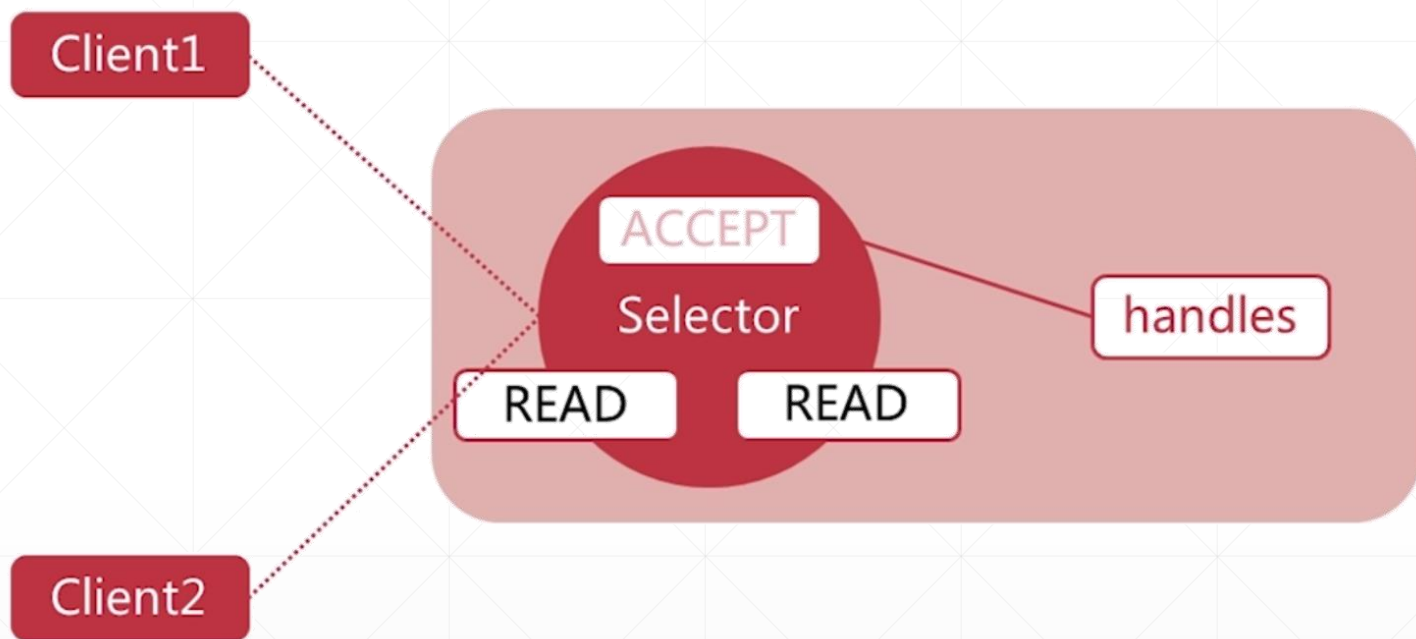


用Selector监控多个Channel



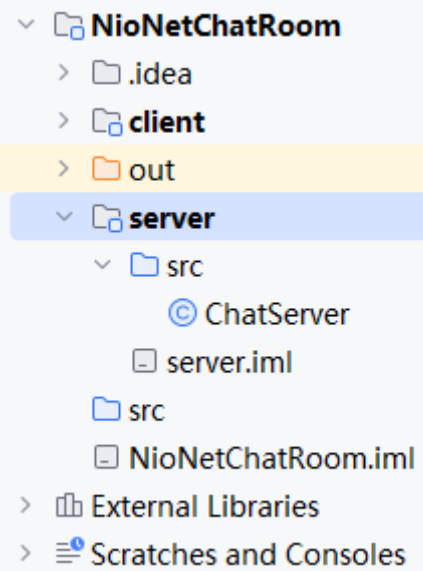
在一个线程里处理多个Channel I/O

NIO网络聊天室基本原理



一个客户端对应一个Channel，Selector监听它们是否有数据到达，并触发Read事件，在此事件的响应代码中，完成数据的读取与转发操作。

服务端代码



定义一些常量及字段

2 usages

```
public class ChatServer {
```

1 usage

```
private static final int DEFAULT_PORT = 8888;
```

1 usage

```
private static final String QUIT = "quit";
```

2 usages

```
private static final int BUFFER = 1024;
```

4 usages

```
private ServerSocketChannel server;
```

9 usages

```
private Selector selector;
```

3 usages

```
private int port;
```

+

```
public ChatServer() { this(DEFAULT_PORT); }
```

2 usages

+

```
public ChatServer(int port) { this.port = port; }
```

Start方法启动服务器

```
public static void main(String[] args) {  
    ChatServer chatServer = new ChatServer(port: 7777);  
    chatServer.start();  
}
```

一些辅助方法

//获取用户信息

4 usages

```
private String getClientName(SocketChannel client) {  
    return "客户端[" + client.socket().getPort() + "];"  
}
```

//检查用户是否发来了“quit”消息

1 usage

```
private boolean readyToQuit(String msg) {  
    return QUIT.equals(msg);  
}
```

//释放资源

1 usage

```
private void close(Closeable closable) {  
    if (closable != null) {  
        try {  
            closable.close();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

核心代码-1

```
private void start() {  
    try {  
        //开启服务端Socket通道  
        server = ServerSocketChannel.open();  
        //配置为非阻塞状态  
        server.configureBlocking(false);  
        //监听指定的端口  
        server.socket().bind(new InetSocketAddress(port));  
        //打开选择器  
        selector = Selector.open();  
        //将用于监听的通道注册到Selector, 指明对Accept(即接收客户端连接请求) 感兴趣  
        server.register(selector, SelectionKey.OP_ACCEPT);  
        System.out.println("启动服务器, 监听端口: " + port + "...");  
    }  
}
```

```
while (true) {  
    //在此阻塞等待客户端的连接请求  
    selector.select();  
    //如果有连接请求, 提取出相关的信息数据项 (即SelectionKey)  
    Set<SelectionKey> selectionKeys = selector.selectedKeys();  
    //依次处理信息数据项  
    for (SelectionKey key : selectionKeys) {  
        // 处理数据项  
        handles(key);  
    }  
    //本轮数据项已经处理完毕, 清空集合, 继续循环, 阻塞等待下次连接  
    selectionKeys.clear();  
}  
  
} catch (IOException e) {  
    e.printStackTrace();  
} finally {  
    close(selector);  
}  
}
```


核心代码-2

```
1 usage
private void handles(SelectionKey key) throws IOException {
    // ACCEPT事件 - 和客户端建立了连接
    if (key.isAcceptable()) {
        //接收用户连接请求
        handleUserConnection(key);
    }
    // READ事件 - 客户端发送了消息
    else if (key.isReadable()) {
        //接收用户消息, 并转发给其他用户
        readAndForwardMessage(key);
    }
}
```



接收用户连接请求

```
1 usage
private void handleUserConnection(SelectionKey key) throws IOException {
    ServerSocketChannel server = (ServerSocketChannel) key.channel();
    //获取用于与客户端进行数据传播的通道
    SocketChannel client = server.accept();
    //配置为非阻塞状态
    client.configureBlocking(false);
    //向选择器注册, 表明此通道将用于接收数据, 对OP_READ事件感兴趣
    client.register(selector, SelectionKey.OP_READ);
    System.out.println(getClientName(client) + "已连接");
}
```

接收用户消息，并转发给其他用户

接收消息

```
private void readAndForwardMessage(SelectionKey key) throws IOException {  
    SocketChannel client = (SocketChannel) key.channel();  
    //从通道中接收信息  
    String fwdMsg = receive(client);  
    if (fwdMsg.isEmpty()) {  
        // 如果消息没取出来，表明客户端异常，以后就不再需要接收信息了  
        key.cancel();  
        //立即从此轮处理中返回  
        selector.wakeup();  
    } else {  
        //消息取出来了  
        System.out.println(getClientName(client) + ":" + fwdMsg);  
        //向其它在线用户转发消息  
        forwardMessage(client, fwdMsg);  
        // 检查用户是否退出  
        if (readyToQuit(fwdMsg)) {  
            key.cancel();  
            selector.wakeup();  
            System.out.println(getClientName(client) + "已断开");  
        }  
    }  
}
```

```
//用于接收消息的缓冲区  
4 usages  
private ByteBuffer rBuffer = ByteBuffer.allocate(BUFFER);  
  
//消息的接收  
1 usage  
private String receive(SocketChannel client) throws IOException {  
    rBuffer.clear();  
    while (client.read(rBuffer) > 0) ;  
    rBuffer.flip();  
    return String.valueOf(charset.decode(rBuffer));  
}
```

转发消息

//用于转发消息使用的ByteBuffer

5 usages

```
private ByteBuffer wBuffer = ByteBuffer.allocate(BUFFER);
```

//使用UTF8解码与编码消息，以支持中文

2 usages

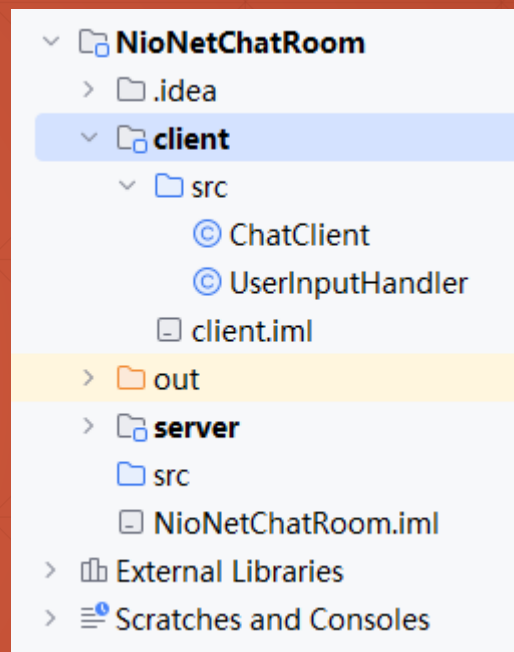
```
private Charset charset = StandardCharsets.UTF_8;
```

//转发消息

1 usage

```
private void forwardMessage(SocketChannel client, String fwdMsg) throws IOException {  
    for (SelectionKey key : selector.keys()) {  
        Channel connectedClient = key.channel();  
        //排除用于监听连接的Server通道  
        if (connectedClient instanceof ServerSocketChannel) {  
            continue;  
        }  
        //消息只转发给其他用户 (排除发消息的那个用户)  
        if (key.isValid() && !client.equals(connectedClient)) {  
            wBuffer.clear();  
            wBuffer.put(charset.encode(str: getClientName(client) + ":" + fwdMsg));  
            wBuffer.flip();  
            while (wBuffer.hasRemaining()) {  
                ((SocketChannel) connectedClient).write(wBuffer);  
            }  
        }  
    }  
}
```

客户端代码



核心代码-1

```
public class ChatClient {  
    1 usage  
    private static final String DEFAULT_SERVER_HOST = "127.0.0.1";  
    1 usage  
    private static final int DEFAULT_SERVER_PORT = 8888;  
    1 usage  
    private static final String QUIT = "quit";  
    2 usages  
    private static final int BUFFER = 1024;  
  
    2 usages  
    private String host;  
    2 usages  
    private int port;  
    5 usages  
    private SocketChannel client;  
  
    8 usages  
    private Selector selector;  
    2 usages  
    private Charset charset = StandardCharsets.UTF_8;
```

```
private void start() {  
    try {  
        //创建Socket通道  
        client = SocketChannel.open();  
        //配置为非阻塞模式  
        client.configureBlocking(false);  
        //创建Selector对象  
        selector = Selector.open();  
        //注册通道, 声明对连接事件 (OP_CONNECT) 感兴趣  
        client.register(selector, SelectionKey.OP_CONNECT);  
        //按照预先设定好的IP地址和端口连接服务器  
        client.connect(new InetSocketAddress(host, port));  
        while (true) {  
            selector.select(); //等待服务器响应连接请求  
            Set<SelectionKey> selectionKeys = selector.selectedKeys();  
            for (SelectionKey key : selectionKeys) {  
                handles(key);  
            }  
            selectionKeys.clear();  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    } catch (ClosedSelectorException e) {  
        // 用户正常退出  
    } finally {  
        close(selector);  
    }  
}
```

核心代码-2

消息的接收

```
4 usages
private ByteBuffer rBuffer = ByteBuffer.allocate(BUFFER);
//接收消息
1 usage
private String receive(SocketChannel client) throws IOException {
    rBuffer.clear();
    while (client.read(rBuffer) > 0);
    rBuffer.flip();
    return String.valueOf(charset.decode(rBuffer));
}
```

```
//处理通道事件
1 usage
private void handles(SelectionKey key) throws IOException {
    // CONNECT事件 - 连接就绪事件
    if (key.isConnectable()) {
        SocketChannel client = (SocketChannel) key.channel();
        if (client.isConnectionPending()) {
            client.finishConnect();
            // 处理用户的输入
            new Thread(new UserInputHandler(chatClient: this)).start();
        }
        //注册通道, 表明对读数据感兴趣
        client.register(selector, SelectionKey.OP_READ);
    }
    // READ事件 - 服务器转发消息
    else if (key.isReadable()) {
        SocketChannel client = (SocketChannel) key.channel();
        String msg = receive(client);
        if (msg.isEmpty()) {
            // 服务器异常
            close(selector);
        } else {
            System.out.println(msg);
        }
    }
}
```

消息的发送

```
private ByteBuffer wBuffer = ByteBuffer.allocate(BUFFER);
//发送消息
1 usage
public void send(String msg) throws IOException {
    if (msg.isEmpty()) {
        return;
    }
    wBuffer.clear();
    wBuffer.put(charset.encode(msg));
    wBuffer.flip();
    while (wBuffer.hasRemaining()) {
        client.write(wBuffer);
    }
    // 检查用户是否准备退出
    if (readyToQuit(msg)) {
        close(selector);
    }
}
```

```
public class UserInputHandler implements Runnable {
    3 usages
    private ChatClient chatClient;
    1 usage
    public UserInputHandler(ChatClient chatClient) {
        this.chatClient = chatClient;
    }
    @Override
    public void run() {
        try {
            // 等待用户输入消息
            BufferedReader consoleReader =
                new BufferedReader(new InputStreamReader(System.in));
            while (true) {
                String input = consoleReader.readLine();
                // 向服务器发送消息
                chatClient.send(input);
                // 检查用户是否准备退出
                if (chatClient.readyToQuit(input)) {
                    break;
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```


示例运行截图

客户端

```
Windows PowerShell
版权所有 (C) Microsoft Corporation

安装最新的 PowerShell, 了解新功

PS D:\test> java ChatClient
Hello!
客户端[6785]:大家好!
客户端[6785]:我来自北京。
I come from USA.
quit
PS D:\test> |
```

```
Windows PowerShell
版权所有 (C) Microsoft Corporation

安装最新的 PowerShell, 了解新功能和

PS D:\test> java ChatClient
客户端[8737]:Hello!
大家好!
我来自北京。
客户端[8737]:I come from USA.
客户端[8737]:quit
|
```

服务端

```
Windows PowerShe
PS D:\test> java ChatServer
启动服务器, 监听端口: 7777...
客户端[6785]已连接
客户端[8737]已连接
客户端[8737]:Hello!
客户端[6785]:大家好!
客户端[6785]:我来自北京。
客户端[8737]:I come from USA.
客户端[8737]:quit
客户端[8737]已断开
|
```